

```
if (i1-i2 == 0.0)
```

which might never be satisfied in floating point arithmetic. Accordingly, we implement the test with an explicit ϵ .

The other problem you need to take care of is when an interval gets subdivided so small that it contains no interior machine-representable point. You then need to terminate the recursion and alert the user that the full accuracy might not have been attained. In the case where the points in an interval are supposed to be $\{a, m = (a + b)/2, b\}$, you can test for $m \leq a$ or $b \leq m$.

The lowest order integration method in the Gander-Gautschi method is the four-point Gauss-Lobatto quadrature (cf. §4.6)

$$\int_{-1}^1 f(x) dx = \frac{1}{6} [f(-1) + f(1)] + \frac{5}{6} \left[f\left(-\frac{1}{\sqrt{5}}\right) + f\left(\frac{1}{\sqrt{5}}\right) \right] \quad (4.7.5)$$

This formula, which is exact for polynomials of degree 5, is used to compute I_2 . To reuse these function evaluations in computing I_1 , they find the seven-point Kronrod extension,

$$\begin{aligned} \int_{-1}^1 f(x) dx = & \frac{11}{210} [f(-1) + f(1)] + \frac{72}{245} \left[f\left(-\sqrt{\frac{2}{3}}\right) + f\left(\sqrt{\frac{2}{3}}\right) \right] \\ & + \frac{125}{294} \left[f\left(-\frac{1}{\sqrt{5}}\right) + f\left(\frac{1}{\sqrt{5}}\right) \right] + \frac{16}{35} f(0) \end{aligned} \quad (4.7.6)$$

whose degree of exactness is nine. The formulas (4.7.5) and (4.7.6) get scaled from $[-1, 1]$ to an arbitrary subinterval $[a, b]$.

For I_s , Gander and Gautschi find a 13-point Kronrod extension of equation (4.7.6), which lets them reuse the previous function evaluations. The formula is coded into the routine below. You can think of this initial 13-point evaluation as a kind of Monte Carlo sampling to get an idea of the order of magnitude of the integral. But if the integrand is smooth, this initial evaluation will itself be quite accurate already. The routine below takes advantage of this.

Note that to reuse the four function evaluations in (4.7.5) in the seven-point formula (4.7.6), you can't simply bisect intervals. But dividing into six subintervals works (there are six intervals between seven points).

To use the routine, you need to initialize an `Adapt` object with your required tolerance,

```
Adapt s(1.0e-6);
```

and then call the `integrate` function:

```
ans=s.integrate(func,a,b);
```

You should check that the desired tolerance could be met:

```
if (s.out_of_tolerance)
    cout << "Required tolerance may not be met" << endl;
```

The smallest allowed tolerance is 10 times the machine precision. If you enter a smaller tolerance, it gets reset internally. (The routine will work using the machine precision itself, but then it usually just takes lots of function evaluations for little additional benefit.)

The implementation of the `Adapt` object is given in a Webnote [2].

Adaptive quadrature is no panacea. The above routine has no special machinery to deal with singularities other than to refine the neighboring intervals. By using

suitable schemes for I_1 and I_2 , one can customize an adaptive routine to deal with a particular kind of singularity (cf. [3]).

CITED REFERENCES AND FURTHER READING:

- Gander, W., and Gautschi, W. 2000, "Adaptive Quadrature — Revisited," *BIT* vol. 40, pp. 84–101.[1]
 Numerical Recipes Software 2007, "Implementation of Adapt," *Numerical Recipes Webnote No. 4*, at <http://www.nr.com/webnotes?4> [2]
 Piessens, R., de Doncker-Kapenga, E., Überhuber, C., and Kahaner, D. 1983 *QUADPACK, A Subroutine Package for Automatic Integration* (New York: Springer). Software at <http://www.netlib.org/quadpack>. [3]
 Davis, P.J., and Rabinowitz, P. 1984, *Methods of Numerical Integration*, 2nd ed., (Orlando, FL: Academic Press), Chapter 6.

4.8 Multidimensional Integrals

Integrals of functions of several variables, over regions with dimension greater than one, are *not easy*. There are two reasons for this. First, the number of function evaluations needed to sample an N -dimensional space increases as the N th power of the number needed to do a one-dimensional integral. If you need 30 function evaluations to do a one-dimensional integral crudely, then you will likely need on the order of 30000 evaluations to reach the same crude level for a three-dimensional integral. Second, the region of integration in N -dimensional space is defined by an $N - 1$ dimensional boundary that can itself be terribly complicated: It need not be convex or simply connected, for example. By contrast, the boundary of a one-dimensional integral consists of two numbers, its upper and lower limits.

The first question to be asked, when faced with a multidimensional integral, is, can it be reduced analytically to a lower dimensionality? For example, so-called *iterated integrals* of a function of one variable $f(t)$ can be reduced to one-dimensional integrals by the formula

$$\int_0^x dt_n \int_0^{t_n} dt_{n-1} \cdots \int_0^{t_3} dt_2 \int_0^{t_2} f(t_1) dt_1 = \frac{1}{(n-1)!} \int_0^x (x-t)^{n-1} f(t) dt \quad (4.8.1)$$

Alternatively, the function may have some special symmetry in the way it depends on its independent variables. If the boundary also has this symmetry, then the dimension can be reduced. In three dimensions, for example, the integration of a spherically symmetric function over a spherical region reduces, in polar coordinates, to a one-dimensional integral.

The next questions to be asked will guide your choice between two entirely different approaches to doing the problem. The questions are: Is the shape of the boundary of the region of integration simple or complicated? Inside the region, is the integrand smooth and simple, or complicated, or locally strongly peaked? Does the problem require high accuracy, or does it require an answer accurate only to a percent, or a few percent?

If your answers are that the boundary is complicated, the integrand is *not* strongly peaked in very small regions, and relatively low accuracy is tolerable, then your problem is a good candidate for *Monte Carlo integration*. This method is very straightforward to program, in its cruder forms. One needs only to know a region with simple boundaries that *includes* the complicated region of integration, plus a method of determining whether a random point is inside or outside the region of integration. Monte Carlo integration evaluates the function at a random sample of points and estimates its integral based on that random sample. We will discuss it in more detail, and with more sophistication, in Chapter 7.

If the boundary is simple, and the function is very smooth, then the remaining approaches, breaking up the problem into repeated one-dimensional integrals, or multidimensional Gaussian quadratures, will be effective and relatively fast [1]. If you require high accuracy, these approaches are in any case the *only* ones available to you, since Monte Carlo methods are by nature asymptotically slow to converge.

For low accuracy, use repeated one-dimensional integration or multidimensional Gaussian quadratures when the integrand is slowly varying and smooth in the region of integration, Monte Carlo when the integrand is oscillatory or discontinuous but not strongly peaked in small regions.

If the integrand *is* strongly peaked in small regions, and you know where those regions are, break the integral up into several regions so that the integrand is smooth in each, and do each separately. If you don't know where the strongly peaked regions are, you might as well (at the level of sophistication of this book) quit: It is hopeless to expect an integration routine to search out unknown pockets of large contribution in a huge N -dimensional space. (But see §7.9.)

If, on the basis of the above guidelines, you decide to pursue the repeated one-dimensional integration approach, here is how it works. For definiteness, we will consider the case of a three-dimensional integral in x, y, z -space. Two dimensions, or more than three dimensions, are entirely analogous.

The first step is to specify the region of integration by (i) its lower and upper limits in x , which we will denote x_1 and x_2 ; (ii) its lower and upper limits in y at a specified value of x , denoted $y_1(x)$ and $y_2(x)$; and (iii) its lower and upper limits in z at specified x and y , denoted $z_1(x, y)$ and $z_2(x, y)$. In other words, find the numbers x_1 and x_2 , and the functions $y_1(x)$, $y_2(x)$, $z_1(x, y)$, and $z_2(x, y)$ such that

$$\begin{aligned} I &\equiv \iiint dx dy dz f(x, y, z) \\ &= \int_{x_1}^{x_2} dx \int_{y_1(x)}^{y_2(x)} dy \int_{z_1(x, y)}^{z_2(x, y)} dz f(x, y, z) \end{aligned} \quad (4.8.2)$$

For example, a two-dimensional integral over a circle of radius one centered on the origin becomes

$$\int_{-1}^1 dx \int_{-\sqrt{1-x^2}}^{\sqrt{1-x^2}} dy f(x, y) \quad (4.8.3)$$

Now we can define a function $G(x, y)$ that does the innermost integral,

$$G(x, y) \equiv \int_{z_1(x, y)}^{z_2(x, y)} f(x, y, z) dz \quad (4.8.4)$$

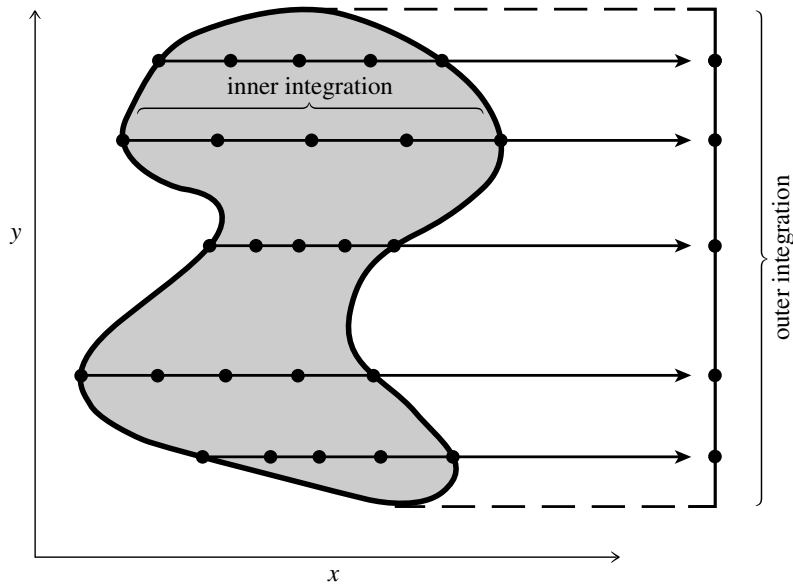


Figure 4.8.1. Function evaluations for a two-dimensional integral over an irregular region, shown schematically. The outer integration routine, in y , requests values of the inner, x , integral at locations along the y -axis of its own choosing. The inner integration routine then evaluates the function at x locations suitable to it. This is more accurate in general than, e.g., evaluating the function on a Cartesian mesh of points.

and a function $H(x)$ that does the integral of $G(x, y)$,

$$H(x) \equiv \int_{y_1(x)}^{y_2(x)} G(x, y) dy \quad (4.8.5)$$

and finally our answer as an integral over $H(x)$

$$I = \int_{x_1}^{x_2} H(x) dx \quad (4.8.6)$$

In an implementation of equations (4.8.4) – (4.8.6), some basic one-dimensional integration routine (e.g., `qgaus` in the program following) gets called recursively: once to evaluate the outer integral I , then many times to evaluate the middle integral H , then even more times to evaluate the inner integral G (see Figure 4.8.1). Current values of x and y , and the pointers to the user-supplied functions for the integrand and the boundaries, are passed “over the head” of the intermediate calls through member variables in the three functors defining the integrands for G , H and I .

```
quad3d.h  struct NRf3 {
           Doub xsav,ysav;
           Doub (*func3d)(const Doub, const Doub, const Doub);
           Doub operator()(const Doub z)  The integrand  $f(x,y,z)$  evaluated at fixed  $x$  and
           {                                $y$ .
               return func3d(xsav,ysav,z);
           }
       };
```

```

struct NRf2 {
    NRf3 f3;
    Doub (*z1)(Doub, Doub);
    Doub (*z2)(Doub, Doub);
    NRf2(Doub zz1(Doub, Doub), Doub zz2(Doub, Doub)) : z1(zz1), z2(zz2) {}
    Doub operator()(const Doub y)    This is  $G$  of eq. (4.8.4).
    {
        f3.ysav=y;
        return qgaus(f3,z1(f3.xsav,y),z2(f3.xsav,y));
    }
};

struct NRf1 {
    Doub (*y1)(Doub);
    Doub (*y2)(Doub);
    NRf2 f2;
    NRf1(Doub yy1(Doub), Doub yy2(Doub), Doub z1(Doub, Doub),
        Doub z2(Doub, Doub)) : y1(yy1),y2(yy2), f2(z1,z2) {}
    Doub operator()(const Doub x)    This is  $H$  of eq. (4.8.5).
    {
        f2.f3.xsav=x;
        return qgaus(f2,y1(x),y2(x));
    }
};

template <class T>
Doub quad3d(T &func, const Doub x1, const Doub x2, Doub y1(Doub), Doub y2(Doub),
    Doub z1(Doub, Doub), Doub z2(Doub, Doub))
Returns the integral of a user-supplied function func over a three-dimensional region specified
by the limits x1, x2, and by the user-supplied functions y1, y2, z1, and z2, as defined in (4.8.2).
Integration is performed by calling qgaus recursively.
{
    NRf1 f1(y1,y2,z1,z2);
    f1.f2.f3.func3d=func;
    return qgaus(f1,x1,x2);
}

```

Note that while the function to be integrated can be supplied either as a simple function

```
Doub func(const Doub x, const Doub y, const Doub z);
```

or as the equivalent functor, the functions defining the boundary can only be functions:

```

Doub y1(const Doub x);
Doub y2(const Doub x);
Doub z1(const Doub x, const Doub y);
Doub z2(const Doub x, const Doub y);

```

This is for simplicity; you can easily modify the code to take functors if you need to.

The Gaussian quadrature routine used in quad3d is simple, but its accuracy is not controllable. An alternative is to use a one-dimensional integration routine like qtrap, qsimp or qromb, which have a user-definable tolerance eps. Simply replace all occurrences of qgaus in quad3d by qromb, say.

Note that multidimensional integration is likely to be very slow if you try for too much accuracy. You should almost certainly increase the default eps in qromb from 10^{-10} to 10^{-6} or bigger. You should also decrease JMAX to avoid a lot of waiting around for an answer. Some people advocate using a smaller eps for the inner quadrature (over z in our routine) than for the outer quadratures (over x or y).

CITED REFERENCES AND FURTHER READING:

- Stroud, A.H. 1971, *Approximate Calculation of Multiple Integrals* (Englewood Cliffs, NJ: Prentice-Hall).[1]
- Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §7.7, p. 318.
- Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), §6.2.5, p. 307.
- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nist.gov/aands>, equations 25.4.58ff.